

MILESTONE 1

Kelompok: Unisoft

Nomor Kelompok: G03

List Anggota Kelompok:

- 1. Bayu Palamarta Wirawan - 13525085**
- 2. Muhammad Fauzi Muharam - 13525091**
- 3. Muhammad Pandu Pulunggana - 13525059**
- 4. Kevin Lincoln Hutabarat - 13525101**

PROGRESS

1. COMPLETED TASKS

No	Task	Penanggung Jawab (NIM)
1	startGame	13525085
2	cekInfo	13525085
3	ambilKartu	13525091
4	lihatKartu	13525059
5	mainKartu	13525101
6	lihatCommand	13525101

2. ONGOING TASKS

No	Task	Penanggung Jawab (NIM)
1	draw_two	
2	wild_draw_four	
3	skip	

4	reverse	
---	---------	--

3. UNSTARTED TASKS

No	Task	Penanggung Jawab (NIM)
1	tantang	
2	tangkap	
3	uni	
4	endGame	
5	saveGame	
6	loadGame	

RANCANGAN FAKTA

No	Fakta	Deskripsi
1.	<code>statusEfek(off).</code>	statusEfek digunakan untuk situasi ketika pemain terkena drawtwo atau wilddrawfour
2.	<code>get_element([Element _], 0, Element).</code>	Permulaan fungsi get_element
3.	<code>input_pemain(-1, _).</code>	Basis input nama pemain
4.	<code>buat_list(_, 0, []).</code>	Basis buat_list
5.	<code>start_kartu(0).</code>	Basis start_kartu
6.	<code>count([],0).</code>	Basis count
7.	<code>cetakpemain(0).</code>	Basis cetakpemain
8.	<code>printKartu([], []).</code>	Basis printKartu

9.	<code>printKoma([]).</code>	Basis printKoma
10.	<code>tampilkanKartu([], [], _).</code>	Basis tampilkanKartu
11.	<code>kartu([0,1,2,3,4,5,6,7,8,9,'skip','reverse','drawtwo','wild','wilddrawfour'], ['merah','kuning','hijau','biru']).</code>	Menyatakan semua kemungkinan nilai kartu
12.	<code>get([Element _], 0, Element).</code>	Basis get untuk file1
13.	<code>ambil_7_kali([], [], 0).</code>	Basis ambil_7_kali
14.	<code>top_card_sebelumnya(a,b).</code>	Basis top_card_sebelumnya untuk tantang
15.	<code>insert_head(H, [], [H]). insert_head(H, T, [H T]).</code>	Memasukan head dalam array
16.	<code>removeListIdx([_ T1], [_ T2], T1, T2, 0).</code>	Basis removeListIdx
17.	<code>validasiwarna('merah'). validasiwarna('biru'). validasiwarna('kuning'). validasiwarna('hijau').</code>	Memeriksa 4 kemungkinan warna kartu
18.	<code>cekAngka(_, _). cekSkip(_, _). cekReverse(_, _). cekWild(_). cekDrawTwo(_, _). cekDrawFour(_).</code>	Basis pengecekan kartu
19.	<code>listkosong([], [], 1). listkosong([_ _], [], 0). listkosong([], [_ _], 0). listkosong([_ _], [_ _], 0).</code>	Menentukan apakah 2 list itu kosong
20.	<code>jumlahElemenList([], 0).</code>	Basis jumlahElementList

21	ceksalah(1).	Basis ceksalah
22	cekTantang(_, Count, Count).	Basis cekTantang
23	cekUni(_, Count, Count).	Basis cekUni

RANCANGAN RULE

No	Rule	Deskripsi
1.	<pre>get_element([_ Tail], Index, Element) :- Index > 0, NI is Index - 1, get_element(Tail, NI, Element).</pre>	Mencari elemen di list dengan index tertentu
2.	<pre>cek_pemain(X, X) :- X >= 2, X <= 4, !.</pre>	Kondisi jumlah pemain sesuai
3.	<pre>cek_pemain(_, Y) :- write('Mohon masukan angka antara 2-4: '), read(Z), nl, cek_pemain(Z, Y).</pre>	Kondisi jumlah pemain tidak sesuai
4.	<pre>cek_nama(X, X) :- \+ listpemain(_, X, _, _), !. cek_nama(_, Y) :- write('Nama sudah digunakan. Masukan nama lain: '), read(Z), nl, cek_nama(Z, Y).</pre>	Mengecek apakah ada nama duplikat
5.	<pre>input_pemain(X, Y) :- X >= 0, write('Masukan nama pemain '),</pre>	Memasukan nama pemain dalam dynamic

	<pre> Y1 is Y - X, write(Y1), write(': '), read(Z), cek_nama(Z, Z1), assertz(listpemain(Y1, Z1, [], [])), X1 is X - 1, input_pemain(X1, Y). </pre>	
6.	<pre> buat_list(X, X, [X T]) :- X > 0, X1 is X - 1, buat_list(X1, X1, T). </pre>	Membuat list baru untuk urutan pemain
7.	<pre> start_kartu(XValid):- XValid > 0, listpemain(XValid, Nama, _, _), ambil_7_kali(KartuBaru, WarnaBaru, 7), retract(listpemain(XValid, Nama, _, _)), assertz(listpemain(XValid, Nama, KartuBaru, WarnaBaru)), X1 is XValid - 1, start_kartu(X1). </pre>	Memilih 7 kartu bebas untuk setiap pemain
8.	<pre> printurutan([T]) :- listpemain(T, Nama, _, _), write(Nama), write(' '), nl. </pre>	Print nama pemain giliran terakhir

9.	<pre> printurutan([H T]) :- listpemain(H, Nama, _, _), write(Nama), write(' - '), printurutan(T). </pre>	Print nama pemain giliran pertama hingga kedua terakhir
10.	<pre> startGame :- write('Masukan jumlah pemain: '), read(X), cek_pemain(X, XValid), nl, X1 is XValid - 1, input_pemain(X1, XValid), assertz(jumlahPemain(XValid)) , buat_list(XValid, XValid, R), permutation(R, R1), !, assertz(urutanGiliran(R1)), write('Urutan pemain: '), printurutan(R1), start_kartu(XValid), write('Setiap pemain mendapatkan 7 kartu acak'), nl, ambil_kartu_top(A, B), assertz(top_card(A, B)), /*print */ write('Kartu discard top: '), write(B), write('-'), write(A), nl, write('Giliran '), get_element(R1, 0, C), listpemain(C, N, _, _), </pre>	Memulai permainan

	<code>write(N), write('.'),nl.</code>	
11.	<code>count([_ T], X) :- count(T, X1), X is X1 + 1.</code>	Banyak anggota dalam array
12.	<code>cetakpemain(X):- X > 0, X1 is X - 1, cetakpemain(X1), listpemain(X, Y, Z, _), write('Nama pemain '), write(X), write(': '), write(Y), nl, count(Z, N), write('Jumlah kartu : '), write(N), nl, nl.</code>	Print semua pemain untuk cekInfo
13.	<code>cekInfo :- top_card(X, Y), !, write('Kartu discard top: '), write(X), write('-'), write(Y), write(.),nl,nl, write('Urutan pemain: '), urutanGiliran(Z), printurutan(Z), nl, nl, jumlahPemain(P), cetakpemain(P).</code>	Mengecek jumlah kartu tiap orang
14.	<code>ambilKartu:- /*lihat kartu paling atas, kalo +2, ambil 2 kartu. perlu akses urutan turn dan tumpukan paling atas*/ statusEfek(on), urutanGiliran([X _]),</code>	Ambil kartu apabila terkena drawtwo

	<pre> listpemain(X, _, _, _), top_card(Kartu, _), Kartu = drawtwo, !, ambilKartu(X, 2). </pre>	
15.	<pre> ambilKartu:- /*jika +4 maka ambil 4 kartu*/ statusEfek(on), urutanGiliran([X _]), listpemain(X, _, _, _), top_card(Kartu, _), Kartu = wilddrawfour, !, ambilKartu(X, 4). </pre>	Ambil kartu apabila terkena wilddrawfour
16.	<pre> ambilKartu:- /*jika bukan +2/+4 maka ambil 1 kartu*/ urutanGiliran([X _]), listpemain(X, _, _, _), ambilKartu(X, 1). </pre>	Ambil kartu biasa
17.	<pre> ambilKartu(Id, JumlahKartu) :- listpemain(Id, Nama, K_Lama, W_Lama), ambilKartu(JumlahKartu, K_Baru, W_Baru), write(Nama), write(' mendapatkan '), write(JumlahKartu), write(' dengan rincian:'), printKartu(K_Baru, W_Baru), nl, append(K_Lama, K_Baru, K_Total), append(W_Lama, W_Baru, </pre>	Ambil kartu sesuai id pemain dan jumlah kartu yang diambil

	<pre> W_Total), retract(listpemain(Id, Nama, K_Lama, W_Lama)), assertz(listpemain(Id, Nama, K_Total, W_Total)), matikan_status_efek, putarGiliran, urutanGiliran(UrutanBaru), write('Urutan pemain: '), printurutan(UrutanBaru), nl, UrutanBaru = [NextId _], listpemain(NextId, NamaNext, _, _), write('Giliran '), write(NamaNext), write(' '), nl. </pre>	
18.	<pre> matikan_status_efek :- statusEfek(on), retract(statusEfek(on)), assertz(statusEfek(off)), !. </pre>	Menonaktifkan statusEfek supaya pemain setelah pemain yang terkena drawtwo/wilddrawfour tidak harus mengambil kartu
19.	<pre> ambilKartu(0, [], []) :- !. ambilKartu(N, [K SisaK], [W SisaW]) :- N > 0, ambil_kartu_acak(K, W), N1 is N - 1, ambilKartu(N1, SisaK, SisaW). </pre>	Ambil kartu acak untuk warna dan jenis
20.	<pre> printKartu([K SisaK], [W SisaW]) :- write(W), write('-'), </pre>	Print kartu untuk ambilKartu

	<pre> write(K), printKoma(SisaK), printKartu(SisaK, SisaW). </pre>	
21.	<pre> printKoma([_ _]) :- write(', '). </pre>	Mencetak koma untuk printKartu
22.	<pre> putarGiliran :- retract(urutanGiliran([H T])) , append(T, [H], UrutanBaru), assertz(urutanGiliran(UrutanB aru)). </pre>	Menukar giliran setelah giliran orang pertama selesai
23.	<pre> lihatKartu :- urutanGiliran([IdPemain _]), listpemain(IdPemain, _, ListKartu, ListWarna), write('Berikut kartu yang anda miliki. '), nl, tampilkanKartu(ListKartu, ListWarna, 1). </pre>	Memperlihatkan kartu milik pemain giliran saat ini
24.	<pre> tampilkanKartu([Kartu TK], [Warna TW], N) :- write(N), write('. '), write(Warna), write('-'), write(Kartu), nl, N1 is N + 1, tampilkanKartu(TK, TW, N1). </pre>	Menampilkan kartu untuk lihatKartu
25	<pre> get([_ Tail], Index, Element) </pre>	Mendapatkan elemen tertentu di list

	<code>:- Index > 0, NI is Index - 1, get(Tail, NI, Element).</code>	dalam file1
26	<code>ambil_kartu_acak(X, Y):- random(0, 15, P), kartu(A, _), get(A, P, X), X = 'wild', !, Y = 'hitam'.</code>	Ambil kartu acak jika mendapatkan wild
27	<code>ambil_kartu_acak(X, Y):- random(0, 15, P), kartu(A, _), get(A, P, X), X = 'wilddrawfour', !, Y = 'hitam'.</code>	Ambil kartu acak jika mendapatkan wilddrawfour
28	<code>ambil_kartu_acak(X, Y) :- random(0, 13, P), random(0, 4, Q), kartu(A, B), get(A, P, X), get(B, Q, Y).</code>	Ambil kartu acak untuk kasus lainnya
29	<code>ambil_kartu_top(X,Y) :- random(0, 10, P), random(0, 4, Q), kartu(A, B), get(A, P, X), get(B, Q, Y).</code>	Mengambil kartu paling atas untuk memulai permainan
30	<code>ambil_7_kali([H T], [A B], N) :- N > 0, ambil_kartu_acak(H, A), N1 is N - 1, ambil_7_kali(T, B, N1).</code>	Mengambil kartu 7 kali untuk memulai pertandingan.
31	<code>removeListIdx([H1 T1], [H2 T2], X, Y, Idx):- Idx>0, I is Idx-1, removeListIdx(T1, T2, X1, Y1, I), insert_head(H1,X1,X), insert_head(H2,Y1,Y).</code>	Menghapus elemen dari array

32	<pre>cekKartuValid(K):- urutanGiliran(R1), get_element(R1, 0, C), listpemain(C, _, X, _), jumlahElemenList(X, Sum), S is Sum+1, K<S, K>=1.</pre>	Memeriksa apakah sebuah kartu valid secara angka pilihan
33	<pre>pilihWarna(Y, _):- \+validasiwarna(Y), nl, write('Warna tidak valid, input lagi:'), read(Y1), pilihWarna(Y1, _). pilihWarna(Y, Y):- validasiwarna(Y).</pre>	Memilih warna apabila memainkan kartu wild
34	<pre>cekAngka(X1,Y1):- number(X1), X1>=0, X1<10, putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y1)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl.</pre>	Permainan kartu angka biasa
35	<pre>cekSkip(X1,Y1):- X1 == 'skip', write('Pemain berikutnya kehilangan giliran. '), nl, nl, putarGiliran,</pre>	Permainan kartu skip

	<pre> putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y1)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl. </pre>	
36	<pre> cekReverse(X1,Y1):- X1 == 'reverse', urutanGiliran([H T]), reverse_list([H T],R), retract(urutanGiliran(_)), assertz(urutanGiliran(R)), retract(top_card(_,_)), assertz(top_card(X1,Y1)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl. </pre>	Permainan kartu reverse
37	<pre> cekWild(X1):- X1 == 'wild', write('Pilih warna: '), read(Y2), nl, pilihWarna(Y2,Y3), putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y3)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl. </pre>	Permainan kartu wild

38	<pre>cekDrawTwo(X1,Y1):- X1 == 'drawtwo', retract(statusEfek(_)), assertz(statusEfek(on)), putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y1)), ambilKartu.</pre>	Permainan kartu drawtwo
39	<pre>cekDrawFour(X1):- X1 == 'wilddrawfour', write('Pilih warna: '), read(Y2), nl, pilihWarna(Y2,Y3), retract(statusEfek(_)), assertz(statusEfek(on)), putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y3)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl. cekDrawFour(_).</pre>	Permainan kartu wilddrawfour
40	<pre>cekKartu(A, B, _, Y1):- number(A), A>=0, A<10, B == Y1, !. cekKartu(A, _, X1, _):- number(A), number(X1), A>=0, A<10, A == X1, !.</pre>	Mengecek apakah kriteria kartu sesuai untuk dimainkan

```
cekKartu(A, _, X1, _):-  
    A == 'skip',  
    A == X1,  
    !.
```

```
cekKartu(A, B, _, Y1):-  
    A == 'skip',  
    B == Y1,  
    !.
```

```
cekKartu(A, _, X1, _):-  
    A == 'reverse',  
    A == X1,  
    !.
```

```
cekKartu(A, B, _, Y1):-  
    A == 'reverse',  
    B == Y1,  
    !.
```

```
cekKartu(A, B, X1, Y1):-  
    A == 'drawtwo',  
    A \== X1,  
    B == Y1,  
    !.
```

```
cekKartu(A, B, X1, Y1):-  
    A == 'wild',  
    A \== X1,  
    B == Y1,  
    !.
```

```
cekKartu(A, B, X1, Y1):-  
    A == 'wilddrawfour',  
    A \== X1,  
    B == Y1,  
    !.
```

```
cekKartu(A, _, X1, _):-  
    X1 == 'wild',
```

	<pre> A \== 'wild', !. cekKartu(A, _, X1, _):- X1 == 'wilddrawfour', A \== 'wilddrawfour', !. </pre>	
41	<pre> mainkanKartu(_):- statusEfek(on), write('Perintah tidak valid. '), !, nl. mainkanKartu(K):- \+cekKartuValid(K), write('Kartu tidak valid! '), !, nl. mainkanKartu(K):- urutanGiliran(R1), top_card(A, B), get_element(R1, 0, C), listpemain(C, _, X, Y), cekKartuValid(K), Idx is K-1, get_element(X, Idx, X1), get_element(Y, Idx, Y1), \+cekKartu(A, B, X1, Y1), !, write('Kartu tidak valid! '), nl. mainkanKartu(K):- urutanGiliran(R1), top_card(A, B), get_element(R1, 0, C), </pre>	Memainkan 1 kartu berdasarkan pilihan pengguna

	<pre> listpemain(C, N, X, Y), cekKartuValid(K), Idx is K-1, get_element(X, Idx, X1), get_element(Y, Idx, Y1), cekKartu(A, B, X1, Y1), nl, write(N), write(' memainkan kartu: '), write(Y1), write('-'), write(X1), nl, nl, removeListIdx(X, Y, X2, Y2, Idx), retract(listpemain(C,N,_,_)), assertz(listpemain(C,N,X2,Y2)), cekAngka(X1,Y1), cekSkip(X1,Y1), cekReverse(X1,Y1), cekDrawTwo(X1,Y1), cekWild(X1), cekDrawFour(X1), retract(top_card_sebelumnya(_ ,_)), assertz(top_card_sebelumnya(A ,B)). </pre>	
42	<pre> jumlahElemenList([_ T],Sum):- jumlahElemenList(T,S1), Sum is S1+1. </pre>	Mencari jumlah element di sebuah list
43	<pre> cekSemuaKartu(_,_,_,_ ,N,N): -!, ceksalah(0). cekSemuaKartu(A,B,X,Y,Count,N </pre>	Memeriksa apakah ada setidaknya satu kartu yang bisa dimainkan

	<pre>):- get_element(X,Count,X1), get_element(Y,Count,Y1), \+cekKartu(A,B,X1,Y1), C1 is Count+1, cekSemuaKartu(A,B,X,Y,C1,N). cekSemuaKartu(A,B,X,Y,Count,_):- get_element(X,Count,X1), get_element(Y,Count,Y1), cekKartu(A,B,X1,Y1), !</pre>	
44	<pre>cekMain(_,_,_,_ ,Out):- statusEfek(on), !, Out is 1. cekMain(_,_ ,X,Y,Out):- listkosong(X,Y,1), !, Out is 1. cekMain(A,B,X,Y,Out):- listkosong(X,Y,0), jumlahElemenList(X,N), cekSemuaKartu(A,B,X,Y,0,N), !, write('1. mainkanKartu'), nl, Out is 2. cekMain(_,_,_,_ ,Out):- Out is 1.</pre>	Menentukan apakah aksi utama mainkanKartu bisa dijalankan untuk lihatCommand
45	<pre>cekTantang(A, Count, Out):- A == 'wilddrawfour', statusEfek(on), !, write(Count), write('. tantang'), nl,</pre>	Memeriksa apakah aksiantang bisa dilakukan

	Out is Count+1.	
46	<pre>cekUni(X,Count, Out):- jumlahElemenList(X,Sum), Sum == 2, !, write(Count), write('. uni'), nl, Out is Count+1.</pre>	Memeriksa apakah aksi uni bisa dilakukan
47	<pre>cekTangkap(N,_,N):- !. cekTangkap(I,Count,_):- urutanGiliran(R1), get_element(R1, I, C), listpemain(C, _, X, _), jumlahElemenList(X, Sum), ceksatu(Sum), !, write(Count), write('. tangkap'), nl. cekTangkap(I,Count,N):- I1 is I+1, cekTangkap(I1,Count,N).</pre>	Memeriksa apakah aksi tangkap bisa dilakukan
48	<pre>lihatCommand:- urutanGiliran(R1), top_card(A, B), jumlahPemain(N), get_element(R1, 0, C), listpemain(C, _, X, Y), nl, write('Aksi utama yang tersedia:'), nl, cekMain(A,B,X,Y,Count), write(Count), write('. ambilKartu'), nl, C1 is Count+1, cekTantang(A, C1, O1), cekUni(X,O1,O2),</pre>	Memperlihatkan aksi-aksi yang bisa dilakukan saat ini

	<pre>cekTangkap(1,O2,N), nl, write('Aksi pendukung yang tersedia:'), nl, write('1. lihatCommand'), nl, write('2. lihatKartu'), nl, write('3. cekInfo'), nl.</pre>	
--	---	--